AI Incident Triage Assistant

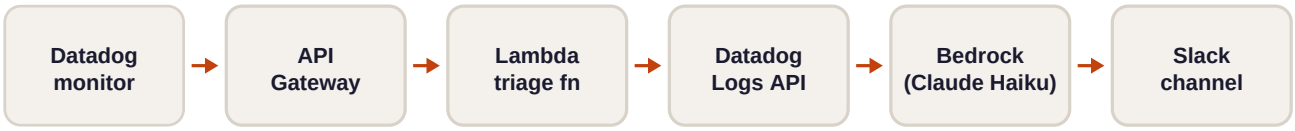
Automated first-pass root cause analysis for production alerts, built on Datadog, AWS Lambda, Amazon Bedrock, and Slack

Overview

When a production alert fires, the first five minutes are almost always the same: someone opens Datadog, pulls up the monitor, scans the surrounding logs, forms a hypothesis, and posts a summary in Slack so the team can decide what to do. That work is repetitive, time-sensitive, and happens at the worst possible moment — while something is actively broken. The AI Incident Triage Assistant automates this first pass. Every time a Datadog monitor fires, the system gathers the relevant log context, asks a large language model for a structured assessment, and posts a formatted triage summary to the team's incident channel before an engineer has even opened a laptop.

The result is that on-call engineers start every incident with a severity estimate, a likely root cause, the list of affected services, and a suggested first action already in front of them — with a direct link back to the triggering monitor for verification. The assistant does not replace human judgment; it eliminates the context-gathering tax that precedes it.

Architecture



The system is fully serverless and adds zero new infrastructure to the existing stack. A Datadog webhook notification channel targets an Amazon API Gateway endpoint, which invokes a single AWS Lambda function. The Lambda enriches the alert with recent log context via the Datadog Logs API, calls Claude Haiku on Amazon Bedrock for analysis, and posts the structured result to Slack via an incoming webhook. End-to-end latency from alert to Slack summary is a matter of seconds.

How It Works

1 Alert fires	A Datadog monitor triggers and sends its webhook payload — alert name, monitor ID, tags, and metadata — to the API Gateway endpoint.
2 Lambda receives the payload	The function parses the alert and identifies the affected service and time window. This forms the seed context for the analysis.

3 Log enrichment	The Lambda queries the Datadog Logs API for the most recent log lines from the affected service in the window around the alert. This step is what makes the output specific rather than generic — the model reasons over the actual error traces, not just the alert title.
4 LLM analysis	Alert metadata and log context are sent to Claude Haiku on Amazon Bedrock with a prompt that requests a strict JSON response: severity (P1/P2/P3), a one-sentence likely root cause, affected services, and a suggested immediate action. Haiku was chosen for its speed and low per-invocation cost, since the function runs on every alert.
5 Slack summary	The JSON is rendered into a Slack Block Kit message with color-coded severity and a direct link back to the Datadog monitor, then posted to the team's incident channel.

Design Decisions

Structured output over free text. The model is instructed to respond in JSON only. This keeps parsing deterministic, makes the Slack formatting reliable, and means malformed responses can be caught and retried rather than posted as noise.

Log enrichment as the core value step. An LLM given only an alert title produces boilerplate. Giving it the surrounding log window is what turns the output into something an on-call engineer will actually trust. The enrichment query is scoped tightly by service and time to control token cost and keep signal-to-noise high.

Small, fast model on Bedrock. Because the function runs on every alert, per-invocation cost and latency dominate model choice. Claude Haiku delivers sub-second-class analysis at a cost that makes the system effectively free to run, and Bedrock keeps the entire pipeline inside the existing AWS account — no new vendor, no data leaving the environment.

Advisory, not autonomous. The assistant proposes; humans decide. Every summary links back to the source monitor so the analysis can be verified in one click, and the severity assessment is framed as an estimate rather than a routing decision.

Impact

Each incident previously began with two to five minutes of manual context-gathering before any real diagnosis could start. The assistant compresses that to seconds, and it does so on every alert, for every engineer, around the clock. Beyond the time savings, it standardizes the first message of every incident — severity, hypothesis, scope, next step — which makes incident channels dramatically easier to scan after the fact and gives newer engineers a consistent model for how to triage.

Skills Demonstrated

This project spans event-driven serverless architecture (API Gateway, Lambda), third-party API integration (Datadog webhooks and Logs API, Slack Block Kit), and applied LLM engineering on Amazon Bedrock — prompt design for structured output, model selection under cost and latency constraints, and defensive handling of nondeterministic responses in a production path. It was conceived, designed, and shipped end-to-end by one engineer inside an existing production observability stack.